

DOCUMENTAÇÃO DE TESTES E QUALIDADE

Projeto: SwagLabs Shopping | Desafio DIO

Esta documentação consolida os artefatos de planejamento e execução de testes funcionais manuais, estruturados sob a ótica de metodologias ágeis e melhores práticas de Quality Assurance (QA). O projeto base utilizado como referência é o **SwagLabs Shopping**.

1. Plano de Fluxo de Trabalho e Ciclo de Vida do Bug

A gestão do ciclo de vida das tarefas e das não conformidades (bugs) é o alicerce para a previsibilidade da entrega. O fluxo abaixo delinea os estados de transição desde a descoberta até a validação das resoluções.

1.1. Fluxo de Trabalho (Development Workflow)

Status	Descrição
TO DO (A Fazer)	A história de usuário ou tarefa foi refinada, estimada e está no Sprint Backlog pronta para ser iniciada.
IN PROGRESS (Em Progresso)	O desenvolvimento técnico da tarefa foi iniciado pela equipe de engenharia.
READY FOR QA (Pronto para QA)	O código foi implementado, passou pelos testes unitários e o ambiente está liberado para validação funcional.
IN TEST (Em Teste)	Os testes manuais, exploratórios e execução dos casos de teste estão ocorrendo.
DONE (Concluído)	Os critérios de aceite foram integralmente atendidos e a funcionalidade validada sem bugs críticos impeditivos.
BLOCKED (Bloqueado)	Avanço impossibilitado por dependências externas, infraestrutura ou bloqueios de negócio.

1.2. Ciclo de Vida do Defeito (Bug Lifecycle)

Status	Descrição da Transição
NEW	Defeito recém-descoberto durante a fase de <i>In Test</i> ou testes exploratórios.
ASSIGNED / OPEN	Defeito triado, priorizado e atribuído a um desenvolvedor para correção.
FIXED / PENDING RETEST	Desenvolvedor aplicou a correção e liberou o pacote para revalidação pelo QA.
RETEST / VERIFIED	QA reexecuta os cenários. Se o comportamento estiver conforme o esperado, o defeito é marcado como Verificado.
REOPENED	Se a correção falhar ou gerar efeitos colaterais na mesma funcionalidade, o defeito retorna para correção.
CLOSED	Ciclo finalizado. Defeito corrigido e validado com sucesso.

2. Histórias de Usuário (User Stories)

As histórias foram elaboradas respeitando o princípio **INVEST** (Independente, Negociável, Valiosa, Estimável, Pequena e Testável) e os **3Cs** (Cartão, Conversa, Confirmação).

US-01: Autenticação no SwagLabs	
Narrativa	Como cliente cadastrado do SwagLabs, Desejo realizar o login utilizando minhas credenciais exclusivas, Para que eu possa acessar a vitrine de produtos e finalizar minhas compras com segurança.
Critérios de Aceite	- O sistema deve validar e-mail e senha correspondentes ao banco de dados. - Em caso de sucesso, redirecionar para a página inicial (vitrine). - Em caso de falha, exibir mensagem genérica: "Credenciais inválidas. Tente novamente." (Prevenção contra enumeração de usuários).

US-02: Gestão do Carrinho de Compras	
Narrativa	Como cliente autenticado, Desejo adicionar e remover produtos do meu carrinho, Para que eu possa controlar os itens antes de seguir para o checkout financeiro.
Critérios de Aceite	- O ícone do carrinho deve atualizar o contador imediatamente ao adicionar um item. - O usuário pode remover o item clicando em "Remove" na vitrine ou dentro da página do carrinho. - Não deve ser possível adicionar um item com estoque zerado.

3. Mind-Map de Casos de Teste (US-01: Autenticação)

Abaixo apresentamos a topologia de testes (representação textual) derivada da arquitetura da US-01, essencial para garantir a cobertura sistêmica antes da escrita detalhada.

[US-01] Autenticação SwagLabs └─ Fluxos de Sucesso (Happy Path) └─ Login com credenciais válidas e ativas └─ Validar redirecionamento para a vitrine └─ Validar geração do token de sessão └─ Fluxos de Exceção (Sad Path) └─ Senha incorreta └─ Validar exibição do aviso de erro genérico └─ Usuário inexistente └─ Validar exibição do aviso de erro genérico └─ Usuário bloqueado (Lockout) └─ Validar mensagem de contatar o suporte └─ Validações de Fronteira / Input └─ Submissão com campos em branco └─ Validar bloqueio do botão "Submit" └─ Injeção de caracteres especiais (SQLi básico) └─ Validar sanitização do formulário └─ Requisitos Não Funcionais (Usabilidade/Segurança) └─ Ocultação de senha (máscara de asteriscos) └─ Responsividade em viewport mobile (320px)

4. Casos de Teste: Técnica Step-by-Step (US-02)

STEP TC-01: Adicionar produto disponível ao carrinho			
Pré-condições: Usuário logado e na página principal. Produto "Backpack" possui estoque > 0. Carrinho atualmente vazio.			
Passo	Ação	Dados de Entrada	Resultado Esperado
1	Localizar o produto desejado na vitrine.	Produto: "Sauce Labs Backpack"	Produto é exibido com botão "Add to cart".
2	Clicar no botão "Add to cart".	N/A	O botão altera sua interface para "Remove".
3	Verificar o ícone do carrinho no cabeçalho superior direito.	N/A	O contador do carrinho é incrementado, refletindo a unidade adicionada.

STEP TC-02: Remover produto pela vitrine principal			
Pré-condições: Usuário logado. Carrinho contém exatamente 1 unidade do produto "Backpack".			
Passo	Ação	Dados de Entrada	Resultado Esperado
1	Localizar o produto previamente adicionado.	Produto: "Sauce Labs Backpack"	O produto exibe o botão indicativo "Remove".
2	Clicar no botão "Remove".	N/A	O botão restaura seu estado original "Add to cart".
3	Verificar o contador no ícone do carrinho.	N/A	O indicador numérico desaparece ou retrocede a zero, consolidando a exclusão.

5. Casos de Teste: Linguagem BDD (Gherkin) (US-01)

BDD BDD-01: Autenticação com sucesso (Happy Path)

Contexto: O sistema de autenticação está operante.

Dado que o usuário está na interface de login do SwagLabs

Quando ele preenche o campo "Username" com "standard_user"

E preenche o campo "Password" com "secret_sauce"

E aciona o botão de submissão do formulário

Então a arquitetura deve validar a sessão do usuário

E redirecioná-lo para a rota da vitrine de produtos ("/inventory.html")

BDD BDD-02: Tentativa de login com usuário bloqueado (Sad Path)

Contexto: Existem diretrizes de logout estipuladas para segurança.

Dado que o usuário se encontra na interface de login do SwagLabs

Quando ele insere as credenciais atreladas a uma conta suspensa (ex: "locked_out_user")

E requisita a autenticação

Então a API deve negar a transação de acesso

E retornar ao front-end a mensagem: "Epic sadface: Sorry, this user has been locked out."